R Cheatsheet

Teaching Week 01 to 12 | The Python sheet carries the same sections in the same order.

The Python cheatsheet has the same sections in the same order, so the other call is found by reading across. The boxes form one script on `heart.csv`, the table Class 01 opens with, run in order from the folder that holds it. A comment that starts with `differs` marks a line whose result is different in the other language, and says why. Rscript prints every top-level value, as the console does.

1 Loading Packages

Install each package once with `install.packages()`. The boxes call `class`, `MASS`, `e1071`, `pROC`, `mclust` and `knitr` as `pkg::fn`, which attaches nothing, so none of their names masks a `dplyr` verb.

```r
library(tidyr)                                  # pivot_longer
library(ggplot2)                                # ggplot, geom_histogram, ggsave
library(dplyr)                                  # filter, mutate, summarise, left_join
library(caret)                                  # confusionMatrix, train
library(rpart)                                  # rpart, printcp, prune
library(rpart.plot)                             # rpart.plot
library(cluster)                                # daisy, silhouette
library(dbscan)                                 # kNNdistplot, dbscan, lof
```

2 Reading and Inspecting a Table

Check the shape, the names and the type of every column before any statistic. Text columns become factors when read with `stringsAsFactors = TRUE`, and `summary()` counts them by level.

```r
heart <- read.csv("heart.csv", stringsAsFactors = TRUE)
dim(heart)                                      # 918 12
head(heart)                                     # the first six rows
str(heart)                                      # type and first values per column
summary(heart)                                  # all columns; factors as counts
table(heart$HeartDisease)
```

3 Summary Statistics and Unusual Values

Both languages divide the variance by $n - 1$ and default to type 7 quartiles, linear interpolation in pandas, so these numbers agree. The 1.5 IQR fence catches the 172 impossible cholesterol readings of 0 and 11 high ones.

```r
x <- heart$Cholesterol                                  # one column, to keep the calls short
mean(x); median(x)                                      # 198.7996 223
var(x); sd(x)                                           # 11964.89 109.3841, divisor n - 1
mean(c(2, NA, 4), na.rm = TRUE)                         # 3; NA without na.rm = TRUE
quantile(x, c(0.25, 0.75), type = 7)                    # 173.25 267
q <- quantile(x, c(0.25, 0.75)); iqr <- q[2] - q[1]
IQR(x); range(x)                                        # 93.75; 0 603
out <- x[x < q[1] - 1.5 * iqr | x > q[2] + 1.5 * iqr]
length(out)                                             # 183 = 172 zeros + 11 high values
length(boxplot.stats(x)$out)                            # 183 too; hinges, not type 7
cor(heart$Age, heart$MaxHR)                             # -0.3820447
```

4 Cleaning and Missing Values

Count the missing values before dropping or filling any. The same rows print one lower in Python, because R counts positions from 1 and pandas labels rows from 0.

```r
d <- heart                                              # clean a copy; heart stays as read
zero <- c("RestingBP", "Cholesterol")                   # 0 is not a possible reading of either
d[zero][d[zero] == 0] <- NA
colSums(is.na(d))                                       # RestingBP 1, Cholesterol 172, others 0
head(which(is.na(d$Cholesterol)), 3)                    # 294 295 296, positions from 1
nrow(d[complete.cases(d), ])                            # 746 rows with nothing missing
d$was_missing <- is.na(d$Cholesterol)                   # flag before filling
d$Cholesterol[d$was_missing] <- median(d$Cholesterol, na.rm = TRUE)
trimws(tolower(c(" North ", "north", "SOUTH")))
as.Date("2026-09-15", format = "%Y-%m-%d")
sum(duplicated(d) | duplicated(d, fromLast = TRUE))
```

5 Reshaping, Encoding, Scaling, Binning, and Sampling

`scale()` divides by the sample standard deviation and `StandardScaler` by the population one, so the scaled values differ by a factor of $\sqrt{918/917}$. Both binning calls give the same counts.

```r
long <- pivot_longer(heart, c(RestingBP, MaxHR), names_to = "measure",
                     values_to = "value", cols_vary = "slowest")
dim(long)                                               # 1836 12, two rows per patient
head(model.matrix(~ ChestPainType - 1, heart), 3)
z <- scale(heart[c("Age", "MaxHR")])                    # differs, divides by the n - 1 sd
attr(z, "scaled:scale")                                 # 9.432617 25.460334
# equal width, then equal frequency from the quartile edges
table(cut(heart$Age, breaks = 4))
q <- quantile(heart$Age, seq(0, 1, 0.25))
table(cut(heart$Age, breaks = q, include.lowest = TRUE))
# differs, as the same seed draws other rows in Python
set.seed(3141); s <- heart[sample(nrow(heart), 100), ]
```

6 Plotting

Give a histogram explicit edges, because the default bins differ and only fixed edges give the same counts in both languages. Each `ggplot()` call starts a new plot.

```
edges <- seq(25, 80, by = 5)                          # the same bins in both languages
ggplot(heart, aes(Age)) + geom_histogram(breaks = edges, closed = "left")
ggplot(heart, aes(Sex, Age)) + geom_boxplot(coef = 1.5)
# differs, as bw = 5 is in bpm while seaborn's bw_method scales each group's sd
ggplot(heart, aes(MaxHR, colour = Sex)) + geom_density(bw = 5)
p <- ggplot(heart, aes(Age, MaxHR, colour = factor(HeartDisease))) +
  geom_point(alpha = 0.7) + facet_wrap(~ Sex) +
  labs(x = "Age (years)", y = "Maximum heart rate (bpm)", colour = "HeartDisease")
dir.create("results", showWarnings = FALSE)           # ggsave() needs the folder
ggsave("results/age-maxhr.pdf", p, width = 7, height = 4)
hist(heart$Age, breaks = edges, right = FALSE)$counts   # 4 17 59 ... 24 7
plot(heart$Age, heart$MaxHR)                          # base R scatter plot
```

7 Transforming and Joining Tables

Check the row count after every join. A key repeated in the lookup table would multiply the rows without an error, and the many-to-one check stops it there.

```
summary_table <- heart |>
  filter(Age >= 40) |>                                # keep the rows that qualify
  select(Age, Sex, MaxHR, HeartDisease) |>            # keep the columns you use
  mutate(band = if_else(Age >= 60, "60+", "40-59")) |>
  group_by(Sex, band) |>                              # one group per Sex and band
  summarise(n = n(), mean_maxhr = mean(MaxHR), .groups = "drop") |>
  arrange(desc(mean_maxhr))                           # sort the summary, largest first
summary_table                                         # 4 rows, F 40-59 first
lookup <- data.frame(ChestPainType = c("ASY", "ATA", "NAP", "TA"),
                     pain = c("none", "atypical", "non-anginal", "typical"))
joined <- left_join(heart, lookup, by = "ChestPainType",
                    relationship = "many-to-one")
stopifnot(nrow(joined) == nrow(heart))                # a left join keeps every row
write.csv(summary_table, "results/summary.csv", row.names = FALSE)
```

8 Reproducible Reports

Render from a fresh session with the command shown, so the report holds what the source computes rather than what an interactive session remembers.

```
# in report.Rmd, The mean age is `r sprintf("%.1f", mean(heart$Age))` years.
sprintf("%.1f", mean(heart$Age))                      # "53.5", what the sentence shows
knitr::kable(summary_table, digits = 3)
# Rscript --vanilla -e "rmarkdown::render('report.Rmd', output_format='pdf_document')"
packageVersion("dplyr")                               # versions, for the reproduction record
```

9 Splitting and Scoring Predictions

The two languages put different rows in the test set, so every score from here on differs between the sheets. Two rules stand in for models, MaxHR below 140 for disease and 220 minus age for MaxHR.

```
num <- c("Age", "MaxHR", "Oldpeak")                    # the predictors from here on
set.seed(3141)
i <- createDataPartition(factor(heart$HeartDisease), p = 0.7, list = FALSE)
tr <- heart[i, ]; te <- heart[-i, ]                    # differs, other rows than Python
pred <- factor(as.integer(te$MaxHR < 140)); obs <- factor(te$HeartDisease)
table(obs, pred)                                       # truth down the rows
# caret puts the predictions down the rows, the transpose of table()
confusionMatrix(pred, obs, positive = "1", mode = "prec_recall")
r <- pROC::roc(response = obs, predictor = -te$MaxHR, levels = c(0, 1), direction = "<")
pROC::auc(r)                                           # 0.7366
hr <- te$MaxHR; p <- 220 - te$Age                      # a rule for maximum heart rate
mean(abs(p - hr))                                      # MAE 31.19
sqrt(mean((p - hr)^2))                                 # RMSE 37.72
postResample(pred = p, obs = hr)                       # its Rsquared, 0.183, is cor(p, hr)^2
R2(p, hr, formula = "traditional")                     # -1.167, 1 - SSres/SStot
```

10 Four Models

Each model learns from the training rows only, scaling included, and is scored on the test rows.

```
form <- factor(HeartDisease) ~ Age + MaxHR + Oldpeak   # outcome ~ predictors
m <- colMeans(tr[num]); s <- apply(tr[num], 2, sd)     # training statistics only
set.seed(3141)                                         # knn() breaks ties at random
k5 <- class::knn(scale(tr[num], m, s), scale(te[num], m, s), tr$HeartDisease, k = 5)
mean(k5 == te$HeartDisease)                            # test accuracy 0.716
fit <- lm(MaxHR ~ Age, data = tr)
fit                                                    # intercept 190.1784, slope -0.9968
mean(abs(predict(fit, te) - te$MaxHR))                 # test MAE 19.05
l <- MASS::lda(form, data = tr)
mean(predict(l, te)$class == te$HeartDisease)          # 0.735
sv <- e1071::svm(form, data = tr)                      # radial kernel, cost 1, scales
mean(predict(sv, te) == te$HeartDisease)               # 0.716
plot(te$MaxHR, predict(fit, te)); abline(0, 1)         # predicted against observed
```

11 Decision Trees and Pruning

`cp` is α over the root error and `ccp_alpha` is α over the training rows on Gini impurity, so neither carries over. By default `rpart` stops at `cp = 0.01` and scikit-learn grows to purity.

```
set.seed(3141)                                         # xval = 10 draws random folds
ctl <- rpart.control(cp = 0, minsplit = 20, minbucket = 7, xval = 10)
fit <- rpart(form, data = tr, method = "class", control = ctl)
printcp(fit)                                           # one row per tree size, with xerror
cp_min <- fit$cptable[which.min(fit$cptable[, "xerror"]), "CP"]
pruned <- prune(fit, cp = cp_min)                      # lowest xerror, at 5 splits
rpart.plot(pruned)
pruned$variable.importance                             # Oldpeak 77.0, MaxHR 48.6, Age 20.3
mean(predict(pruned, te, type = "class") == te$HeartDisease)
```

12 Resampling and Tuning

The best cross-validated score is a maximum over the grid, so it is optimistic. Report the test score with a bootstrap interval, or nested cross-validation.

```
ctrl <- trainControl(method = "cv", number = 5)         # folds stratified by outcome
grid <- data.frame(k = c(1, 5, 9, 15))
set.seed(3141)
kfit <- train(form, data = tr, method = "knn", preProcess = c("center", "scale"),
              tuneGrid = grid, trControl = ctrl)
kfit$bestTune                                           # k = 15
max(kfit$results$Accuracy)                              # 0.744, selected, so not an estimate
hit <- predict(kfit, te) == te$HeartDisease
mean(hit)                                               # 0.705, the estimate (te used once)
acc <- replicate(2000, mean(sample(hit, replace = TRUE)))
quantile(acc, c(0.025, 0.975))                          # 0.655 0.756, a 95% interval
outer <- createFolds(factor(heart$HeartDisease), k = 5, returnTrain = TRUE)
mean(sapply(outer, function(j) {                        # tune inside, score outside
  f <- train(form, data = heart[j, ], method = "knn", tuneGrid = grid,
             preProcess = c("center", "scale"), trControl = ctrl)
  mean(predict(f, heart[-j, ]) == heart$HeartDisease[-j])
}))                                                     # 0.734, nested CV
```

13 Distances and Clustering

Scale first, because a distance is in the units of the columns. With the sample standard deviation in both languages, k-means reaches the same three clusters, numbered differently.

```
Z <- scale(heart[num])                                  # divisor n - 1
D <- dist(Z, method = "euclidean")
g <- daisy(heart[c("Age", "Sex", "ChestPainType")], metric = "gower")
# differs for some k, as R runs Hartigan-Wong and scikit-learn runs Lloyd
set.seed(3141)
sapply(1:8, function(k) kmeans(Z, k, nstart = 50, iter.max = 100)$tot.withinss)
km <- kmeans(Z, centers = 3, nstart = 50, iter.max = 100)
table(km$cluster); km$tot.withinss                      # 238 363 317; 1386.006
s <- silhouette(km$cluster, D); mean(s[, "sil_width"])  # 0.2946317
hc <- cutree(hclust(D, method = "complete"), k = 3)
table(hc)                                               # differs, 296 546 76 (tied distances)
kNNdistplot(Z, minPts = 5)                              # distance to the 4th nearest other row
abline(h = 0.5, lty = 2)                                # eps at the knee
db <- dbscan(Z, eps = 0.5, minPts = 5)
table(db$cluster)                                       # 78 noise (0), then 824 4 7 5
mclust::adjustedRandIndex(km$cluster, hc)               # differs, 0.179
```

14 Outliers

The 1.5 IQR fence is in Section 3. The median and the MAD resist the masking that hides a group of large values from it, and LOF ranks rows by local density, with no universal cut-off.

```
# minPts = 5 counts the point itself, as n_neighbors = 4 does not
lof_score <- lof(Z, minPts = 5)
head(order(lof_score, decreasing = TRUE), 3)            # rows 326 333 353
x <- heart$Cholesterol; r <- abs(x - median(x)) / mad(x)  # MADs from the median
sum(r > 3); head(order(r, decreasing = TRUE), 3)        # 180, 172 of them zeros; 150 617 77
```